



UNIVERSITY OF PISA

Master of Science in Computer Engineering

Foundations of Cybersecurity

TIMESTAMPING SERVICE

Students:

Sebastiano Pala

Eleonora Sgorbini

Contents

1	Introduction	3
1.1	Main Components	3
1.2	Secure Channel	4
1.3	Design Assumption	5
2	Sequence Diagrams	7
2.1	Handshake Protocol	7
2.2	Operations over the secure channel	9
2.2.1	Client Authentication	9
2.2.2	Signature Operation	10
2.2.3	Client Balance Request	12
2.3	Timestamps Verification	13

Chapter 1

Introduction

A Timestamping Service (TSS) is a trusted third-party service that provides a legally valid, cryptographically signed record proving that specific electronic data (documents, code, logs) existed at a precise date and time. Rather than signing the entire data, the server signs a cryptographic hash of the data itself, provided by the requesting user, along with a timestamp, thus inextricably binding a time proof to the document's hash.

After a prior registration and the payment of a fee, the user is provided with a volume of timestamps that can be used to perform signatures. At any time, the user can request to view the balance of available and consumed timestamps or request a new signature, until all timestamps are used.

1.1 Main Components

The system is composed by:

- A single **Server** that uses multi-threading connection to interact with multiple clients simultaneously. It handles the handshake requests to establish a secure communication channel, retrieves the balance of the authenticated user and calculates the signature for the requested document if there are still timestamps available.
- Multiple **Clients** that can execute one of the following operation, after has been authenticated by the server:
 - **Balance:** Show the number of available and consumed timestamps
 - **Sign:** Ask the server to sign a document's hash
 - **Exit:** Close the connection with the server and shutdown

In addition, each client can always perform the **Verify** operation: verifies previously obtained signatures, without interacting with the server, and without an account required.

1.2 Secure Channel

All the interactions between users and the TSS are performed through a secure channel that has to be established previously. The channel, to be considered secure, must fulfill the following requirements:

- **Perfect Forward Secrecy (PFS)**
- **Confidentiality**
- **Authentication**
- **Integrity**
- **Non-Malleability**
- **No-Replay**

Perfect Forward Secrecy

The *Perfect Forward Secrecy (PFS)* has been guaranteed by employing the *ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)* protocol during the handshake phase. Both client and server generate and exchange ephemeral keys for the current session: the shared secret used for the subsequent communication is derived from them. This ensures that, even if the server's long-term private key is compromised in the future, previous sessions and message exchanges will remain secure.

Authentication

Authentication is ensured both during the handshake phase and throughout the actual message communication.

- **Handshake:** The server digitally signs both nonces (the client's and its own) as well as the ephemeral keys. By verifying this signature, the client is able to authenticate the server's identity and detect any Man-in-the-Middle attacks.
- **Message Phase:** Data origin authentication is implicitly guaranteed by the *AES-256-GCM* cipher: since the symmetric session key is known solely to the client and the authenticated server, a successful tag verification proves that the message was generated by the legitimate peer.
- **Application Level:** During the Handshake phase, only the server's identity is authenticated. The client's identity is proven during the message phase, by providing his/her username and password.

Confidentiality

Confidentiality is guaranteed by encrypting all traffic using the *AES-256-GCM* cipher. Once the handshake is completed, the shared secret derived via the ECDHE protocol is used to establish the 256-bit symmetric session keys. The use of this strong encryption standard ensures that the plaintext of the messages remains completely inaccessible to any passive attacker observing the channel.

Integrity

The cipher also guarantees integrity, by generating a 16-byte authentication Tag for each encrypted message. During the decryption phase, by verifying the Tag, both client and server are able to detect any modification performed by an attacker.

Non-Malleability

In addition, the use of this cipher also guarantees the *Non-Malleability* property of the channel: despite the encryption itself being malleable, the Tag verification ensures the detection and rejection of any modified message.

No-Replay

Replay attacks are prevented both in the handshake and during the session.

- **Handshake:** The client generates and sends a nonce, which is then signed by the server, along with all the other exchanged parameters. In order to avoid the risk of replay attacks, the nonce must be fresh and unpredictable. For this reason, nonces are generated using a **CSPRNG** through the `RAND_bytes()` OpenSSL function, guaranteeing unpredictability. To practically eliminate the risk of collisions and ensure freshness, 32-byte nonces are used.
- **Message Phase:** *AAD (Additional Authenticated Data)* is used to guarantee anti-replay protection through sequence numbers. Both the client and the server maintain internal directional counters that are included as AAD when computing the message authentication tag. There are two independent sequence counters (one for client-to-server and one for server-to-client traffic). Because their progression is deterministic, they are never transmitted over the network. Upon receiving a message, the receiver uses its expected sequence number as the AAD during decryption; if the tag verification succeeds, the receiver increments its counter for the next message.

1.3 Design Assumption

The deployment of the application is based on the following assumptions:

- **Registration:** The user has already registered with the TSS and paid the corresponding fee. We assume that all registration processes have been completed

and that the necessary authentication credentials are stored in a JSON file accessible only by the server, along with the balance of available and consumed timestamps. A user's entry inside the file has the following structure:

```
1  {
2      "username" : "seba",
3      "salt" : "d01ed9691ade33d90dd772378ed89b3b",
4      "password" : "32e3a7bf378695fc3e8a5338d3163bcef70267671fde
5          721457c4ae0768911a61",
6      "timestamps_remaining" : 99,
7      "timestamps_consumed" : 1
8  },
9
```

- **Public Keys:** The server is equipped with two long term key pairs, each with its own purpose. The public key used during the handshake phase, and the public key used for signature operations, are assumed to be given and safely stored in the client's local files. In other words, we assume that the client has already obtained the server's public keys.
- **Attacker:** The attacker is assumed to be a classical (non-quantum) adversary with polynomially bounded computational resources. This assumption is strictly necessary since the public-key cryptography utilized in the handshake phase (ECDHE) is not resistant to quantum computing attacks.
- **Attacker's Reach:** The attacker does not have physical or privileged remote access to the server's hardware or operating system. The server environment is considered a trusted entity, ensuring the confidentiality of locally stored sensitive files, such as the long-term private keys and the users' credentials database.

Chapter 2

Sequence Diagrams

2.1 Handshake Protocol

Before any application data is exchanged, the client and server execute a cryptographic handshake to establish a secure communication channel. During this phase, the *ECDHE* protocol is performed in order to securely negotiate a shared secret, while digital signatures are utilized to authenticate the server's identity.

Client Hello

The handshake is initiated by the client, which generates a fresh 32-byte nonce, alongside a new ECDHE ephemeral key pair. The client then constructs and sends a 64-byte **Client Hello** message to the server, consisting of its randomly generated nonce concatenated with its ephemeral public key.



Figure 2.1: Client Hello Message

The adjective "raw" indicates that the key is converted from an `EVP_PKEY` object, as generated by the `EVP_PKEY_Q_keygen()` function of the *OpenSSL library*, into a raw byte array to be transmitted over the channel.

Server Hello

Upon receiving the **Client Hello**, the server generates its own nonce and its ephemeral key pair. It then constructs the **Server Hello** message to send back to the client. The first 64 bytes of this payload contain the server's nonce and its ephemeral public key. To guarantee the server's authenticity and prevent Man-in-the-Middle attacks, these 64 bytes are concatenated with a 64-byte digital signature. This signature is computed using the server's long-term Handshake private key over the combina-

tion of both nonces and both ephemeral public keys, thereby strictly binding the ephemeral parameters to the current session.

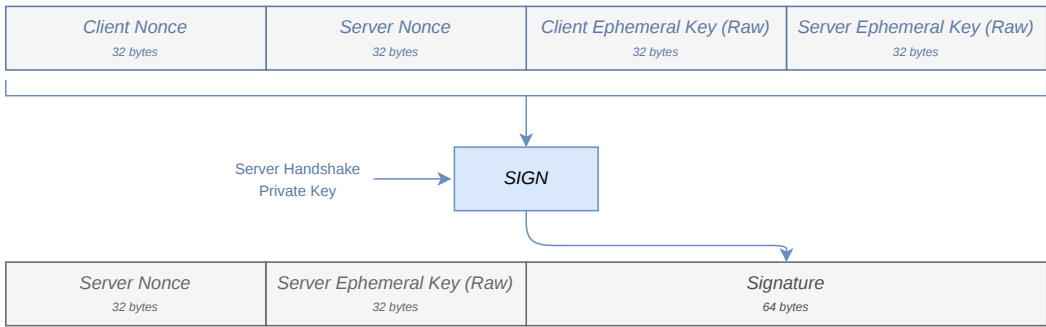
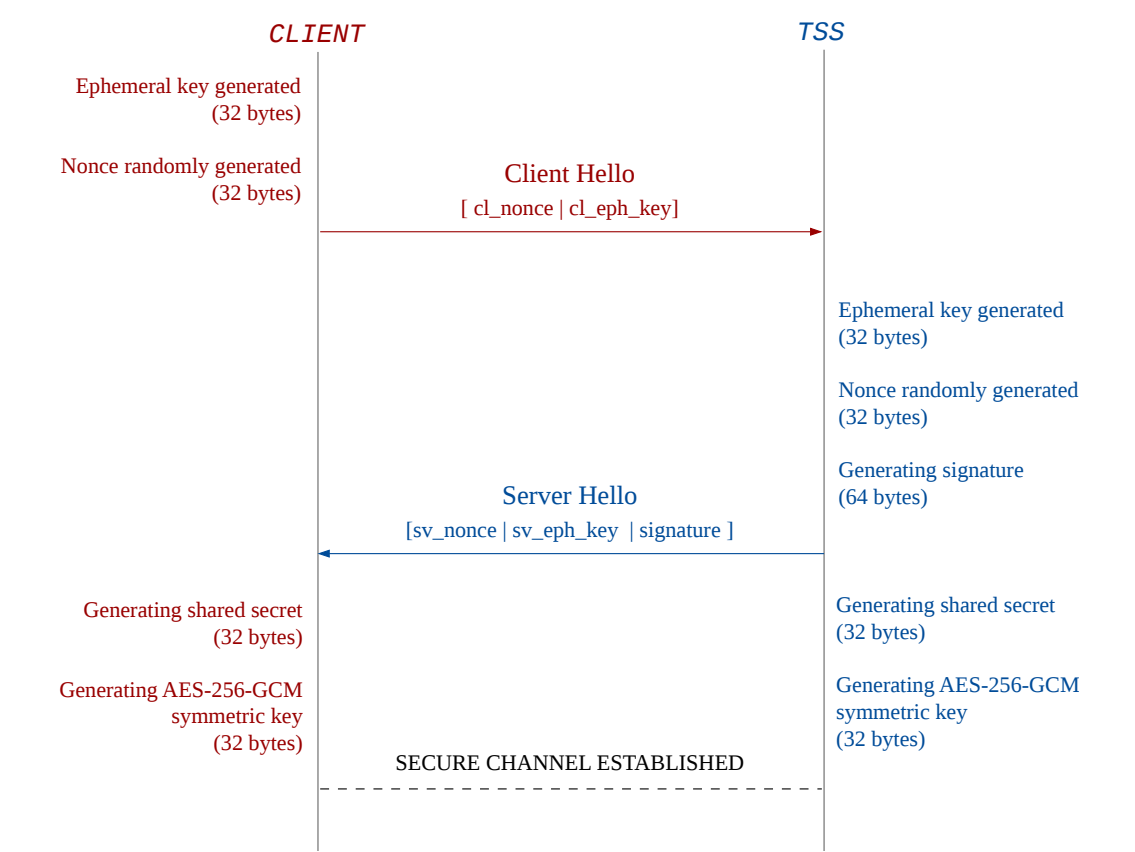


Figure 2.2: Server Hello Message

Once the `Server Hello` message has been received, the client verifies the signature in order to authenticate the server. Now, the client and the server can both calculate the shared secret from the exchanged ephemeral keys. Using this shared secret and the nonces, they derive the symmetric session key for the AES-256-GCM cipher using the HMAC-based Key Derivation Function (HMAC). Once the key has been obtained, the secure channel has been established: the ephemeral key pairs are deleted, and the client and the server can start communicating using symmetric encryption.

Sequence Diagram: Handshake Protocol



2.2 Operations over the secure channel

All subsequent operations involving communication between the client and the server are performed over a secure channel, established during the initial handshake phase. This session channel is protected using the AES-256-GCM symmetric cipher.

The message formats illustrated in the following sections refer exclusively to the plaintext payload. However, considering the specific cipher adopted, it must always be kept in mind that 16 bytes of Initialization Vector (IV) are inserted before the message, and a 16-byte Authentication TAG is appended at the end of the packet transmitted over the network.

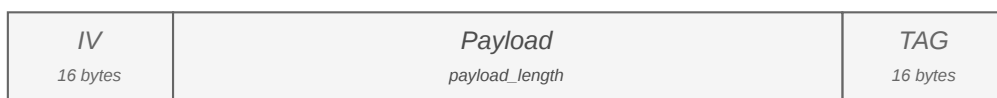


Figure 2.3: Structure AES-256-GCM Message

2.2.1 Client Authentication

Before requesting any operation other than signature verification, the user must authenticate with the server by providing his username and password. The plaintext payload sent to the server is composed of the username and password, both followed by their respective null-termination character.



Figure 2.4: Client Authentication Message

Once the message has been received, the server searches for the username inside its database, which is stored in a specific file (`UserCredentials.json`). If the username does not exist, the authentication fails. Otherwise, the server concatenates the stored salt with the password received from the client, calculates the hash using SHA-256 and compares it with the stored hash. The outcome of the authentication attempt is sent back to the client as a single-byte message, signaling either a positive or negative outcome. If the authentication fails three times in a row, or an error occurs during the authentication, the server returns an `ERROR` status and terminates the connection.

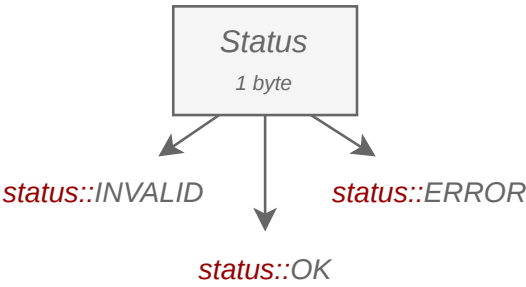


Figure 2.5: Outcome Client Authentication

Sequence Diagram Client Authentication

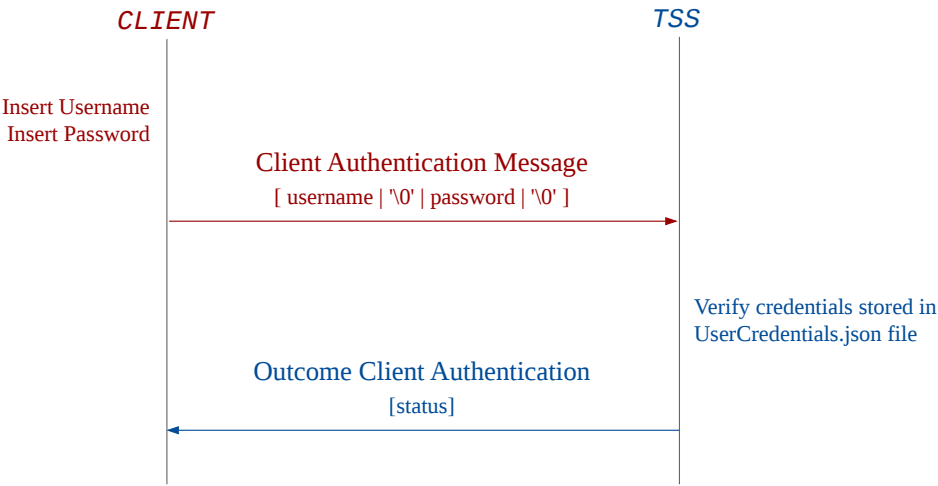


Figure 2.6: Sequence Diagram: Client Authentication

2.2.2 Signature Operation

As specified in the requirements, the user can request to perform the digital signature of a document’s hash. This operation can be requested only after prior authentication with the server, as described in Section 2.2.1. All message exchanges are always protected using the AES-256-GCM symmetric cipher.

Client Sign Request

In the first phase, the user inserts the document’s path via the command line, prompting the client to calculate the hash of the document using SHA-256. The resulting hash, and not the original document, is the target of the signature. The client then sends a SIGN request, along with the computed digest.

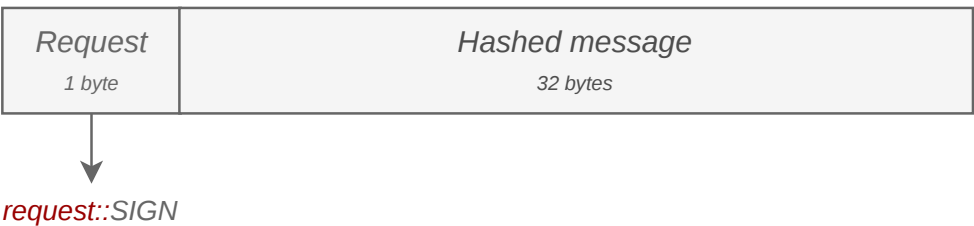


Figure 2.7: Request Hashed Message Signature

Server Sign Response

Upon decrypting the message, the server processes the **SIGN** request. First, it verifies whether the user still has available timestamps; if not, it returns an *INVALID* status. If there are timestamps available, the server generates a current timestamp and builds a payload composed of the client-provided hash, the generated timestamp, and a digital signature computed over both of these fields, using its long term private signature key. The resulting response payload is 105 bytes long, structured as follows:

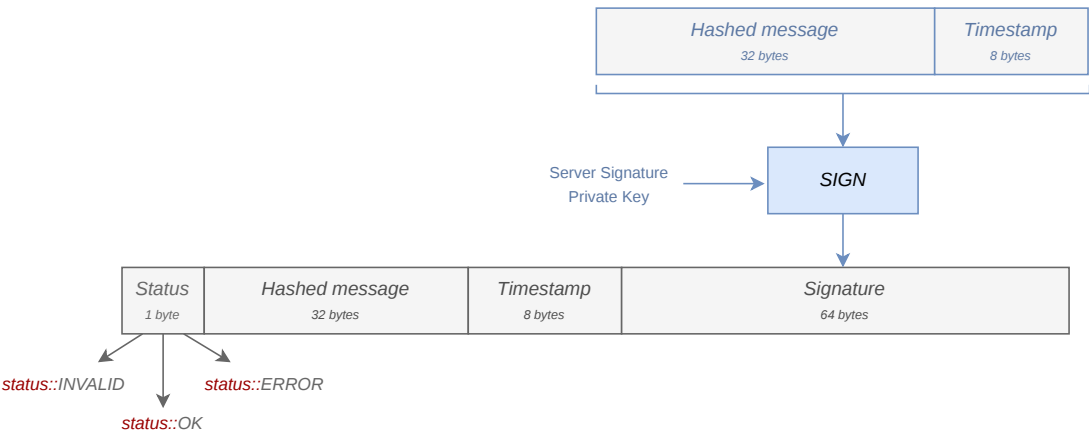


Figure 2.8: Server Signature Response.

Client Verification

Finally, the client evaluates the result of the operation by checking the **Status** field. If the outcome is positive, the client proceeds to validate the signature with the server's long-term public signature key, using the document hash together with and the associated timestamp. If the cryptographic signature is valid, the client displays the formatted timestamp and then stores the newly received signature in a local file, containing all the other previously requested signatures.

Sequence Diagram: Signature Operation

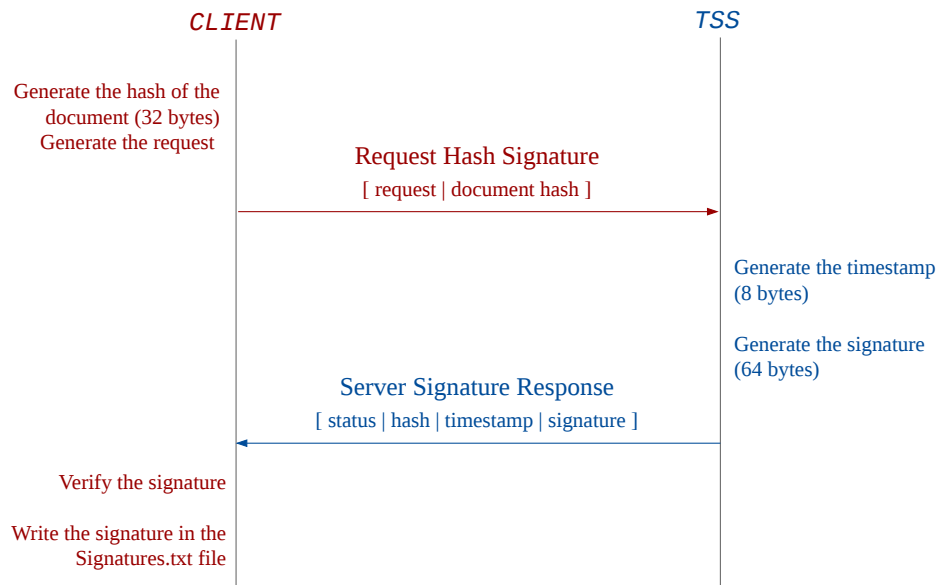


Figure 2.9: Sequence Diagram Signature Operation

2.2.3 Client Balance Request

Once authenticated, the user can access their usage info, by requesting the amount of consumed and available timestamps. The client performs this operation by sending a single-payload, containing the **BALANCE** request type.

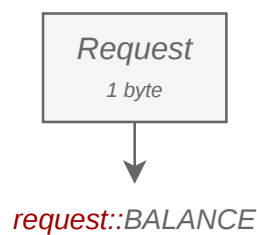


Figure 2.10: Balance Request Message

Server Balance Response

Upon receiving the request, the server accesses its local database to retrieve the timestamp information associated with the authenticated user. It then sends back to the client the operation's outcome (possibly notifying about eventual errors during the database management, for example, any errors during database access), together with the amount of remaining and consumed timestamps.

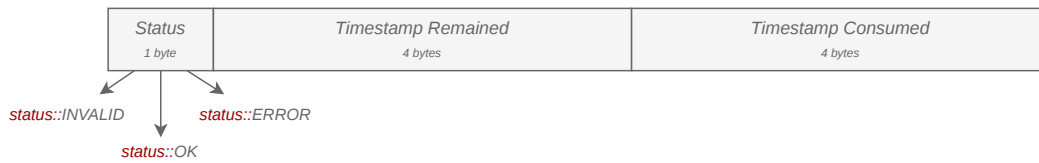


Figure 2.11: Timestamps Balance Message

Client Output Interpretation

Finally, after decrypting the message, if the status is OK, the client reads the timestamp information and prints it on the screen. If the status is INVALID, the client notifies the user about their lack of timestamps.

Sequence Diagram: Balance Request

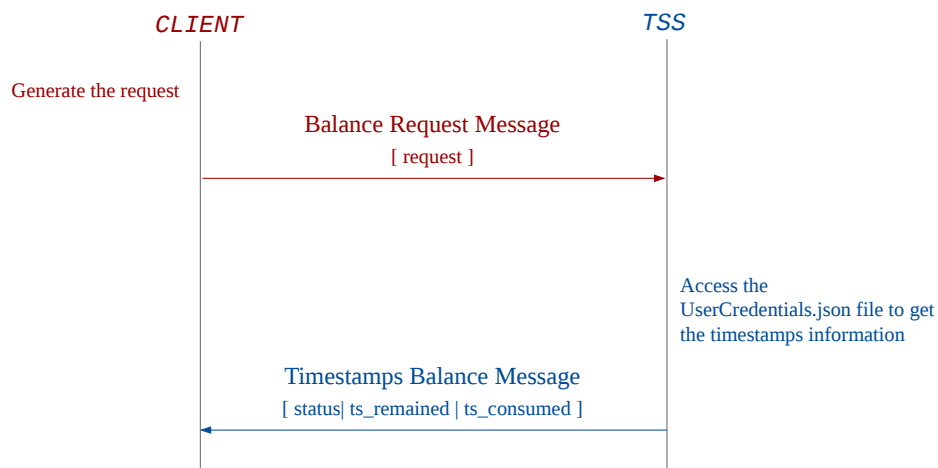


Figure 2.12: Sequence Diagram Balance Request

2.3 Timestamps Verification

Unlike the previous operations, the signature verification process is performed locally by the client, without requiring any communication with the server, and can be performed without requiring an account.

Verification Request

The client prompts the user to provide the path to the document they wish to verify. The client then reads the file and computes its SHA-256 digest on the spot, and uses it to obtain all saved signatures of the requested document, by accessing the local storage file (`Signatures.txt`), which acts as a database containing all the previously saved signature blocks. Each stored block is 104 bytes long and consists of the document's hash (32 bytes), the associated timestamp (8 bytes), and the server's digital signature (64 bytes), and each block is separated by a newline character.

Signatures.txt
HASH TIMESTAMP _ 1 SIGNATURE
HASH TIMESTAMP _ 2 SIGNATURE
HASH TIMESTAMP _ 3 SIGNATURE

Figure 2.13: Signatures File

The client scans this file searching for any entry whose stored hash matches the newly computed hash of the provided document. If one or more matching signatures are found, the client proceeds to verify the cryptographic validity of each one. It extracts the timestamp and the 64-byte digital signature from the stored entry. Then, using the server's long-term public signature key, it verifies the signature against the concatenation of the computed document hash and the extracted timestamp.

Verification Outcome

If the cryptographic verification is successful, it mathematically proves that the server originally signed that exact version of the document at that specific time. The client then formats the 8-byte timestamp into a human-readable local time and displays it to the user. On the contrary, if the verification fails, the client outputs a warning, indicating that the signature or the timestamp might have been modified.

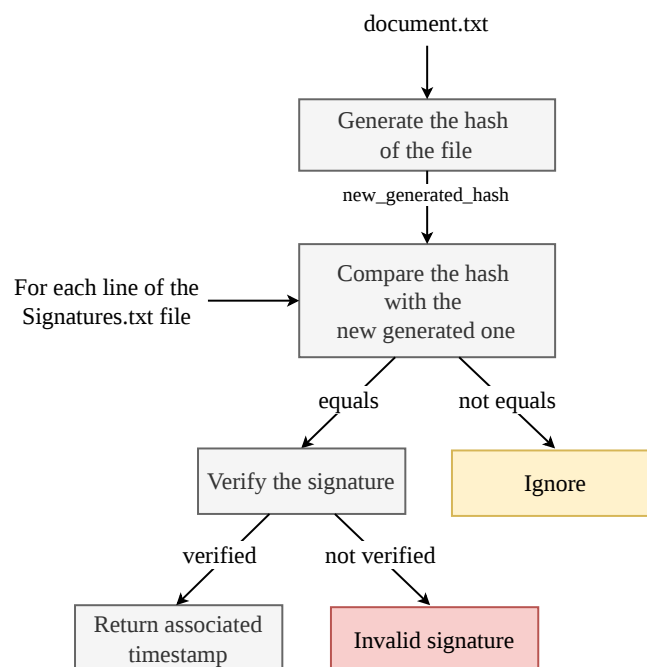


Figure 2.14: Timestamp Verification